

候，浏览器会自动使用指定应用程序来打开。它是一个互联网标准，扩展了电子邮件标准，使其能够支持：非 ASCII 字符文本；非文本格式附件（二进制、声音、图像等）；由多部分（multiple parts）组成的消息体；包含非 ASCII 字符的头信息（Header information）。

因此，电子邮件发送多媒体文件附件时采用 MIME 协议来支持邮件传输。

参考答案

(70) A

试题 (71) ~ (75)

You are developing a server-side enterprise application. It must support a variety of different clients including desktop browsers, mobile browsers and native mobile applications. The application might also expose an API for 3rd parties to consume. It might also (71) with other applications via either web services or a message broker. The application handles requests (HTTP requests and messages) by executing business logic; accessing a database; exchanging messages with other systems; and returning a HTML/JSON/XML (72). There are logical components corresponding to different functional areas of the application.

What's the application's deployment architecture?

Define an architecture that structures the application as a set of (73), collaborating services. This approach corresponds to the Y-axis of the Scale Cube. Each service is:

- Highly maintainable and testable — enables rapid and frequent development and deployment
- Loosely coupled with other services — enables a team to work independently the majority of time on their service(s) without being impacted by changes to other services and without affecting other services
- (74) deployable — enables a team to deploy their service without having to coordinate with other teams
- Capable of being developed by a small team — essential for high productivity by avoiding the high communication head of large teams

Services (75) using either synchronous protocols such as HTTP/REST or asynchronous protocols such as AMQP. Services can be developed and deployed independently of one another. Each service has its own database in order to be decoupled from other services. Data consistency between services is maintained using some particular pattern.

(71) A. integrate

B. coordinate

C. cooperate

D. communicate

(72) A. request

B. response

C. text

D. file

- | | |
|-------------------------|---------------------|
| (73) A. loosely coupled | B. loosely cohesion |
| C. highly coupled | D. highly cohesion |
| (74) A. Dependently | B. Independently |
| C. Coordinately | D. Integratedly |
| (75) A. interoperate | B. coordinate |
| C. communicate | D. depend |

参考译文

假设你在开发一个服务端应用。该应用必须支持各种各样的客户端，包括桌面浏览器、手机浏览器和本地手机应用。应用可能也需要公开部分 API 供第三方使用，还可能与其他应用通过 Web Service 或消息代理（message broker）相集成。应用执行业务逻辑来处理请求（HTTP 请求或者消息）；访问数据库；与其他系统交换消息；并返回 HTML/JSON/XML 类型的响应。有一些逻辑组件对应于应用的不同功能模块。

应用的部署架构是什么？

通过采用 y 轴方向上伸缩立方（Scale Cube）来设计应用的架构，将应用按功能分解为一组松耦合且相互协作的服务的集合。每个服务具有如下特征：

- 高可维护性和可测试性——支持快速、频繁的开发和部署；
- 与其他服务松散耦合——使团队能够在大部分时间独立地工作于其服务上，而不受对其他服务的更改的影响，也不影响其他服务；
- 可独立部署——团队能够不与其他团队协调而部署他们的服务；
- 能够由一个小团队开发——通过避免大型团队的高沟通主管，对高生产力至关重要。

服务间通过 HTTP/REST 等同步协议或 AMQP 等异步协议进行通信。服务可以独立开发和部署。每个服务有其自己的数据库，以便与其他服务解耦。服务之间的数据一致性通过其他特定的模式来维护。

参考答案

(71) A (72) B (73) A (74) B (75) C

第 10 章 软件设计师下午试题分析与解答

试题一（共 15 分）

阅读下列说明和图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某公司欲开发一款二手车物流系统，以有效提升物流成交效率。该系统的主要功能是：

（1）订单管理：系统抓取线索，将车辆交易系统的交易信息抓取为线索。帮买顾问看到有买车线索后，会打电话询问买家是否需要物流，若需要，帮买顾问就将这个线索发起为订单并在系统中存储，然后系统帮助买家寻找物流商进行承运。

（2）路线管理：帮买顾问对物流商的路线进行管理，存储的路线信息包括路线类型、物流商、起止地点。路线分为三种，即固定路线、包车路线、竞拍体系，其中固定路线和包车路线是合约制。包车路线的发车时间由公司自行管理，是订单的首选途径。

（3）合约管理：帮买顾问根据公司与物流商确定的合约，对合约内容进行设置，合约信息包括物流商信息、路线起止城市、价格、有效期等。

（4）寻找物流商：系统根据订单的类型（保卖车、全国购和普通二手车）、起止城市、需要的服务模式（买家接、送到买家等）进行自动派发或以竞拍体系方式选择合适的物流商。即：有新订单时，若为保卖车或全国购，则直接分配到竞拍体系中；否则，若符合固定路线和/或包车路线，系统自动分配给合约物流商，若不符合固定路线和包车路线，系统将订单信息分配到竞拍体系中。竞拍体系接收到订单后，将订单信息推送给有相关路线的物流商，物流商对订单进行竞拍出价，最优报价的物流商中标。最后，给承运的物流商发送物流消息，更新订单的物流信息，给车辆交易系统发送物流信息。

（5）物流商注册：物流商账号的注册开通。

现采用结构化方法对二手车物流系统进行分析与设计，获得如图 1-1 所示的上下文数据流图和图 1-2 所示的 0 层数据流图。

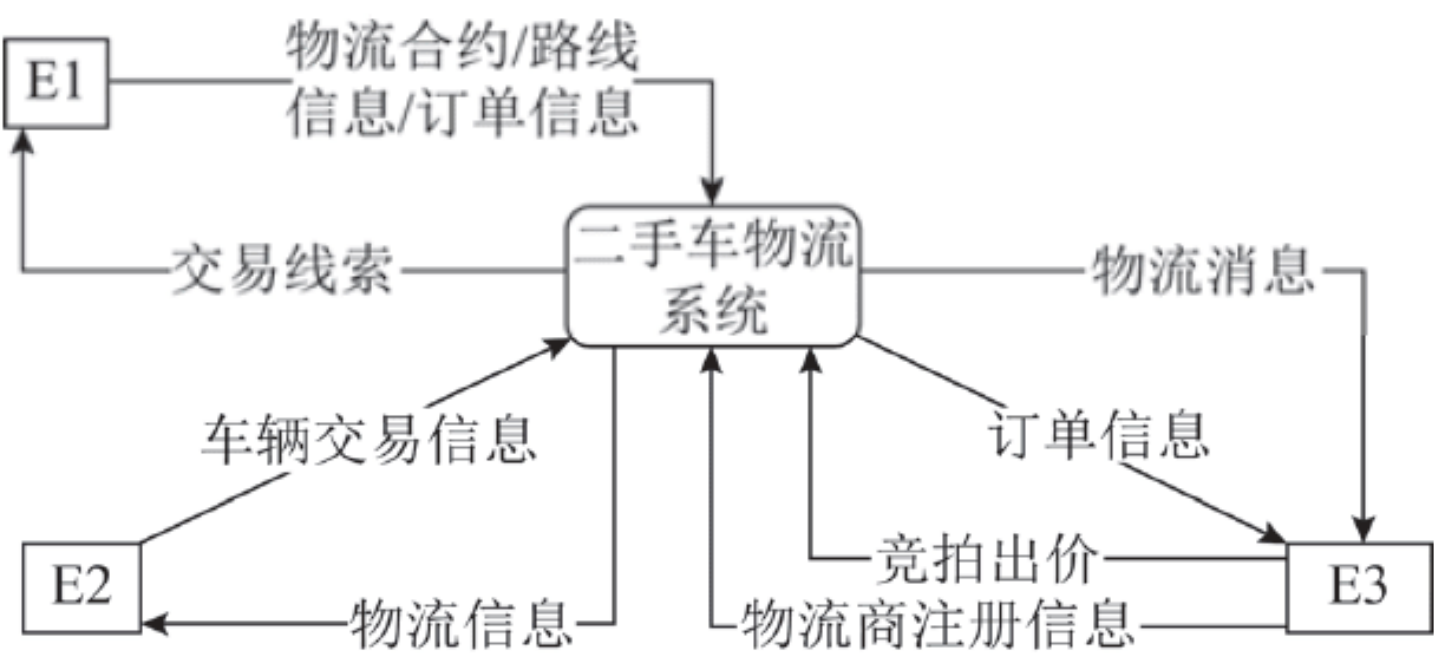


图 1-1 上下文数据流图

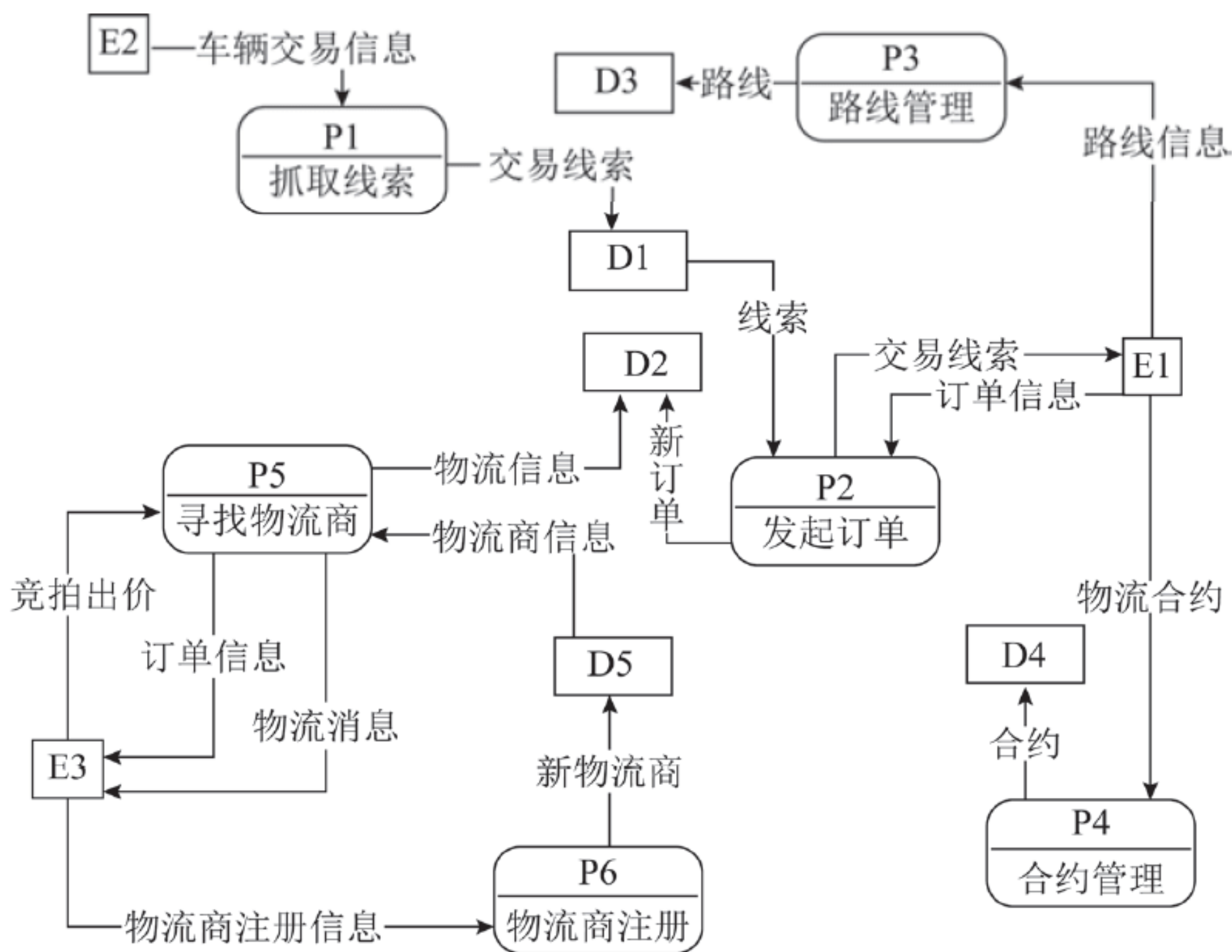


图 1-2 0 层数据流图

【问题 1】（3 分）

使用说明中的词语，给出图 1-1 中的实体 E1~E3 的名称。

【问题 2】（5 分）

使用说明中的词语，给出图 1-2 中的数据存储 D1~D5 的名称。

【问题 3】（4 分）

根据说明和图中术语，补充图 1-2 中缺失的数据流及其起点和终点。

【问题 4】（3 分）

根据说明，采用结构化语言对“P5：寻找物流商”的加工逻辑进行描述。

试题一分析

本题考查采用结构化方法进行软件系统的分析与设计，主要考查利用数据流图（DFD）进行需求分析和建模。DFD 是面向数据流建模的工具，它将系统建模成输入、加工（处理）、输出的模型，即流入软件的数据对象，经由加工的转换，最后以结果数据对象的形式流出软件，并采用自顶向下分层建模进行逐层细化。建立数据字典，对数据流图中的每条数据流、存储、加工和组成数据流或存储的数据项、加工的逻辑再进一步说明。

顶层 DFD（上下文数据流图）建模用于确定系统边界以及系统的输入输出数据，待开发软件系统被看作一个加工，为系统提供输入数据以及接收系统输出数据的是外部实体，外部实体和加工之间的输入输出即为数据流。数据流或者由具体的数据属性构成，或者是由其他数据流构成的组合数据流，用于高层数据流图中。将上下文 DFD 中的加

工分解成多个加工，识别每个加工的输入数据流以及经过加工变换后的输出数据流，建模 0 层 DFD。根据 0 层 DFD 中加工的复杂程度进一步建模加工的内容。根据需求情况可以将数据存储建模在不同层次的 DFD 中。

在建模分层 DFD 时，需要注意加工和数据流的正确使用，一个加工必须既有输入又有输出；数据流须和加工相关，即数据流至少有一头为加工。注意要在绘制下层数据流图时保持父图与子图平衡，即父图中某加工的输入输出数据流必须与其子图的输入输出数据流在数量和名字上相同，或者父图中的一个输入（或输出）数据流对应于子图中几个输入（或输出）数据流的组合数据流。

题目题干描述清晰，易于分析，要求考生细心分析题目中所描述的内容。

【问题 1】

本问题考查的是上下文 DFD，要求确定外部实体。

在上下文 DFD 中，待开发系统“二手车物流系统”作为唯一加工，为这一加工提供输入数据流或者接收其输出数据流的外部实体，涉及车辆交易系统、帮买顾问和物流商，再根据描述相关信息进行对应，对照图 1-1，即可确定 E1 为“帮买顾问”实体，E2 为“车辆交易系统”实体，E3 为“物流商”实体。

【问题 2】

本问题要求确定图 1-2 0 层数据流图中的数据存储。

对照图 1-2 中已经给出的信息，重点分析说明中与数据存储有关的描述。说明（1）中“将车辆交易系统的交易信息抓取为线索”，可知加工“抓取线索”向存储中写入新线索信息，由此可知 D1 为“线索”；再由说明（1）中“帮买顾问就将这个线索发起为订单并在系统中存储”等信息，可知此加工需要向存储中写入新订单信息，由此可知 D2 为“订单”。说明（2）中“帮买顾问对物流商的路线进行管理，存储的路线信息”可知加工“路线管理”向 D3 中写入路线信息，由此可知 D3 为“路线”。说明（3）中合约管理“对合约内容进行设置”等，可知 D4 为“合约”。说明（5）中“选择合适的物流商”和说明（6）中“物流商账号的注册开通”，可知 D5 为“物流商”。

【问题 3】

本问题要求补充缺失的数据流及其起点和终点。

对照图 1-1 和图 1-2 的输入、输出数据流，缺少了从加工到外部实体 E2 的数据流——“物流信息”，说明（4）中，“最后，给承运的物流商发送物流消息”，可知此数据流起点为 P5（寻找物流商），终点为 E2（车辆交易系统）。

再考查题干中的说明判定是否缺失内部的数据流，不难发现图 1-2 中没有完整给出说明（4）加工“寻找物流商”相关的输入数据流。根据描述“根据订单的类型”可知需要从存储 D2（订单）中获取数据；“若符合固定路线和/或包车路线，系统自动分配给合约物流商”说明需要获取存储 D4（路线）和 D3（合约）的信息进行判定然后进行分配。

【问题 4】

本问题要求采用结构化语言描述“P5：寻找物流商”的加工逻辑。结构化语言（如结构化英语）是一种介于自然语言和形式化语言之间的半形式化语言，是自然语言的一个受限子集。

结构化语言没有严格的语法，它的结构通常可分为内层和外层。外层有严格的语法，内层的语法比较灵活，可以接近于自然语言的描述。

（1）外层。用来描述控制结构，采用顺序、选择和重复 3 种基本结构。

①顺序结构。一组祈使语句、选择语句、重复语句的顺序排列。祈使语句是指至少包含一个动词及一个名词，指出要执行的动作及接受动作的对象。

②选择结构。一般用 IF-THEN-ELSE-ENDIF、CASE-OF-ENDCASE 等关键词。

③重复结构。一般用 DO-WHILE-ENDDO、REPEAT-UNTIL 等关键词。

（2）内层。一般采用祈使语句的自然语言短语，使用数据字典中的名词和有限的自定义词，其动词含义要具体，尽量不用形容词和副词来修饰，还可使用一些简单的算法运算和逻辑运算符号。

寻找物流商相关描述中，在有新订单时，明确在不同情况下执行不同行为，最后发送物流消息和更新订单内容。所以，首先是接收新订单；然后根据订单类型采用选择结构；再根据非保卖车或全国购的情况下，根据路线情况不同需执行不同行为，再嵌套一层选择结构；最后顺序发送物流消息和更新订单内容。选择结构可选择 IF-THEN-ELSE-ENDIF，也可以抽象出 CASE 情况，使用 CASE-OF-ENDCASE。

参考答案

【问题 1】

E1：帮买顾问

E2：车辆交易系统

E3：物流商

【问题 2】

D1：线索

D2：订单

D3：路线

D4：合约

D5：物流商

【问题 3】

数据流名称	起点	终点
订单信息	D2 或订单	P5 或寻找物流商
合约信息	D4 或合约	P5 或寻找物流商
路线信息	D3 或路线	P5 或寻找物流商
物流信息	P5 或寻找物流商	E2 或车辆交易系统

【问题 4】

```
接收新订单
IF 是保卖车或全国购
THEN 执行竞拍体系,最优报价物流商中标
ELSE
    IF 订单路线有固定路线和/或包车路线
    THEN 自动派发给物流商
    ELSE 执行竞拍体系,最优报价物流商中标
ENDIF
ENDIF
给物流商发送物流消息
更新订单的物流信息
给车辆交易系统发送物流信息
```

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某公司拟开发一套新入职员工的技能培训管理系统，以便使新员工快速胜任新岗位。该系统的部分功能及初步需求分析的结果如下所述：

- 1. 部门信息包括部门号、名称、部门负责人、电话等，其中部门号唯一标识部门关系中的每一个元组。一个部门有多名员工，但一名员工只属于一个部门；每个部门只有一名负责人，负责部门工作。
- 2. 员工信息包括员工号、姓名、部门号、岗位、基本工资、电话、家庭住址等，其中员工号唯一标识员工关系中的每一个元组；岗位有新入职员工、培训师、部门负责人等；不同的岗位设置不同的基本工资。新入职员工要选择多门课程进行培训，并通过考试取得课程的成绩。一名培训师可以讲授多门课程，一门课程可以由多名培训师讲授。
- 3. 课程信息包括课程号、课程名称、学时等，其中课程号唯一标识课程关系的每一个元组。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图如图 2-1 所示。

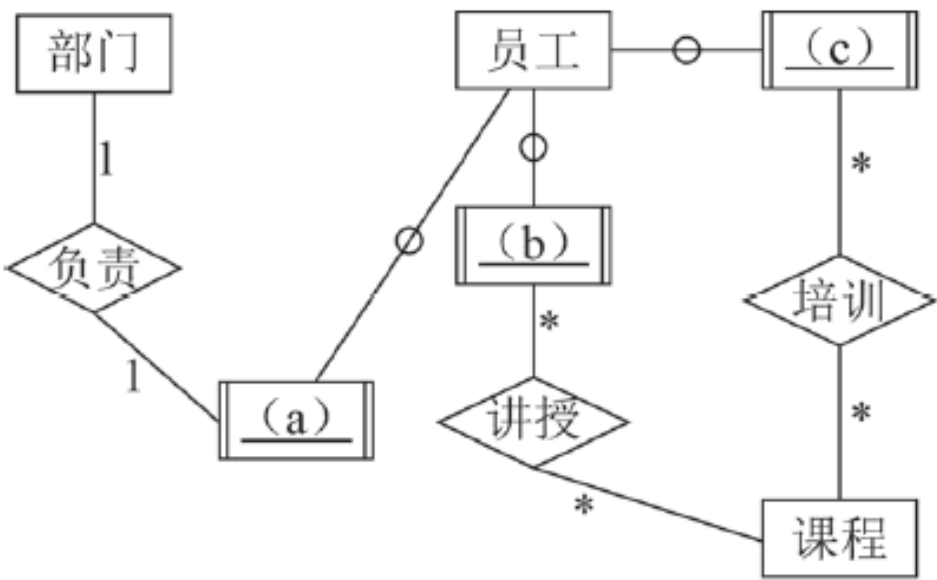


图 2-1 实体联系图

【关系模式设计】

部门(部门号, 部门名, 部门负责人, 电话)

员工(员工号, 姓名, 部门号, (d), 电话, 家庭住址)

课程((e), 课程名称, 学时)

讲授(课程号, 培训师, 培训地点)

培训(课程号, (f))

【问题 1】(5 分)

(1) 补充图 2-1 中的空 (a) ~ (c)。

(2) 图 2-1 中是否存在缺失的联系? 若存在, 则说明所缺失的联系和联系类型。

【问题 2】(3 分)

根据题意, 将关系模式中的空 (d) ~ (f) 补充完整。

【问题 3】(5 分)

(1) 员工关系模式的主键为 (g), 外键为 (h);

(2) 讲授关系模式的主键为 (i), 外键为 (j)。

【问题 4】(2 分)

员工关系是否存在传递依赖? 用 100 字以内文字说明理由。

试题二分析

本题考查数据库系统中实体联系模型 (E-R 模型) 和关系模式设计方面的基础知识。

【问题 1】

根据题意“每个部门只有一名负责人, 负责部门工作”, 那么“负责”联系的两端实体为部门和部门负责人, 故空 (a) 应填写“部门负责人”。

根据题意“一名培训师可以讲授多门课程, 一门课程可以由多个培训师讲授”, 那么“讲课”联系的两端实体为培训师和课程, 故空 (b) 应填写“培训师”。

根据题意“新入职员工至少要选择三门以上的课程进行培训, 并通过考试取得课程的成绩”, 那么“培训”联系的两端实体为新入职员工和课程, 故空 (c) 应填写“新入职员工”。

由于一个部门有多名员工, 但一名员工只对应一个部门。故部门与员工之间的“所属”联系类型为 1:n (或 1.*)。

根据上述分析, 完善图 2-1 所示的实体联系图如图 2-2 所示。

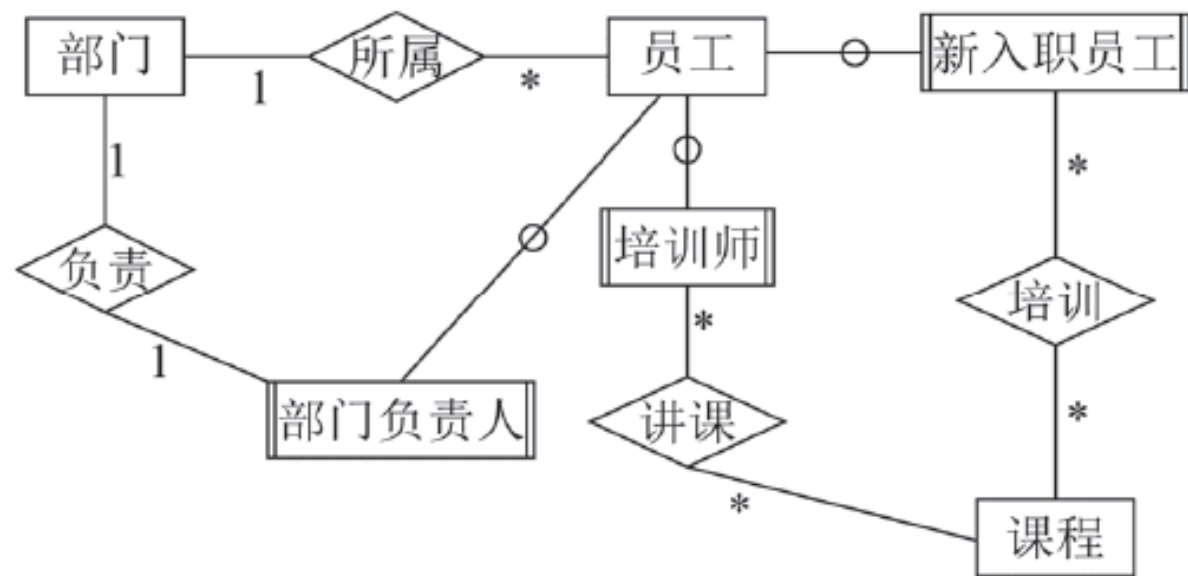


图 2-2 完善的实体联系图

【问题 2】

根据题意员工信息包括：员工号、姓名、部门号、岗位、基本工资、电话、家庭住址。故员工关系模式中的空 (d) 应填写岗位，基本工资。

根据题意课程信息包括：课程号、课程名称、学时，故课程关系中的空 (e) 应填写课程号。

根据题意“新入职员工至少要选择三门以上的课程进行培训，并通过考试取得课程的成绩”，故培训关系的空 (f) 应填写新入职员工，成绩。

【问题 3】

(1) 空 (g) (h) 分析：员工号唯一标识员工关系中的每一个元组，员工关系的主键为员工号。又因为部门号应参照部门关系的部门号，而部门号是部门关系的主键，故员工关系的外键为部门号。

(2) 空 (i) (j) 分析：因为 (课程号，培训师) 唯一确定讲授关系的每一个元组，所以讲授关系的主键为 (课程号，培训师)。又因为课程号应参照课程关系的课程号，培训师应参照员工关系的员工号，故讲授关系存在两个外键分别为课程号和培训师。

【问题 4】

员工关系存在传递依赖。因为，员工号→岗位，岗位→基本工资，根据 Armstrong 公理系统的传递律规则，员工号→基本工资。

参考答案**【问题 1】**

- (1) (a) 部门负责人
- (b) 培训师
- (c) 新入职员工

(2) 存在。(1 分)

图 2-1 中缺失一个部门和员工之间的“所属”联系，联系类型为 1:n。(1 分)

【问题 2】

- (d) 岗位，基本工资
- (e) 课程号
- (f) 新入职员工，成绩

【问题 3】

- (1) (g) 员工号
- (h) 部门号
- (2) (i) (课程号，培训师)
- (j) 课程号 培训师

【问题 4】

存在。(1 分)

因为员工号→岗位、岗位→基本工资，故存在传递依赖“员工号→基本工资”（根据 Armstrong 公理系统的传递律规则）。（1 分）

试题三（共 15 分）

阅读下列说明和图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某牙科诊所拟开发一套信息系统，用于管理病人的基本信息和就诊信息。诊所工作人员包括：医护人员（DentalStaff）、接待员（Receptionist）和办公人员（OfficeStaff）等。系统主要功能需求描述如下：

1. 记录病人基本信息（Maintain patient info）。初次就诊的病人，由接待员将病人基本信息录入系统。病人基本信息包括病人姓名、身份证号、出生日期、性别、首次就诊时间和最后一次就诊时间等。每位病人与其医保信息（MedicalInsurance）关联。

2. 记录就诊信息（Record office visit info）。病人在诊所的每一次就诊，由接待员将就诊信息（OfficeVisit）录入系统。就诊信息包括就诊时间、就诊费用、支付代码、病人支付费用和医保支付费用等。

3. 记录治疗信息（Record dental procedure）。病人在就诊时，可能需要接受多项治疗，每项治疗（Procedure）可能由多位医护人员为其服务。治疗信息包括：治疗项目名称、治疗项目描述、治疗的牙齿和费用等。治疗信息由每位参与治疗的医护人员分别向系统中录入。

4. 打印发票（Print invoices）。发票（Invoice）由办公人员打印。发票分为两种：给医保机构的发票（InsuranceInvoice）和给病人的发票（PatientInvoice）。两种发票内容相同，只是支付的费用不同。当收到治疗费用后，办公人员在系统中更新支付状态（Enter payment）。

5. 记录医护人员信息（Maintain dental staff info）。办公人员将医护人员信息录入系统。医护人员信息包括姓名、职位、身份证号、家庭住址和联系电话等。

6. 医护人员可以查询并打印其参与的治疗项目相关信息（Search and print procedure info）。

现采用面向对象方法开发该系统，得到如图 3-1 所示的用例图和 3-2 所示的初始类图。

【问题 1】（6 分）

根据说明中的描述，给出图 3-1 中 A1~A3 所对应的参与者名称和 U1~U3 所对应的用例名称。

【问题 2】（5 分）

根据说明中的描述，给出图 3-2 中 C1~C5 所对应的类名。

【问题 3】（4 分）

根据说明中的描述，给出图 3-2 中类 C4、C5、Patient 和 DentalStaff 的必要属性。

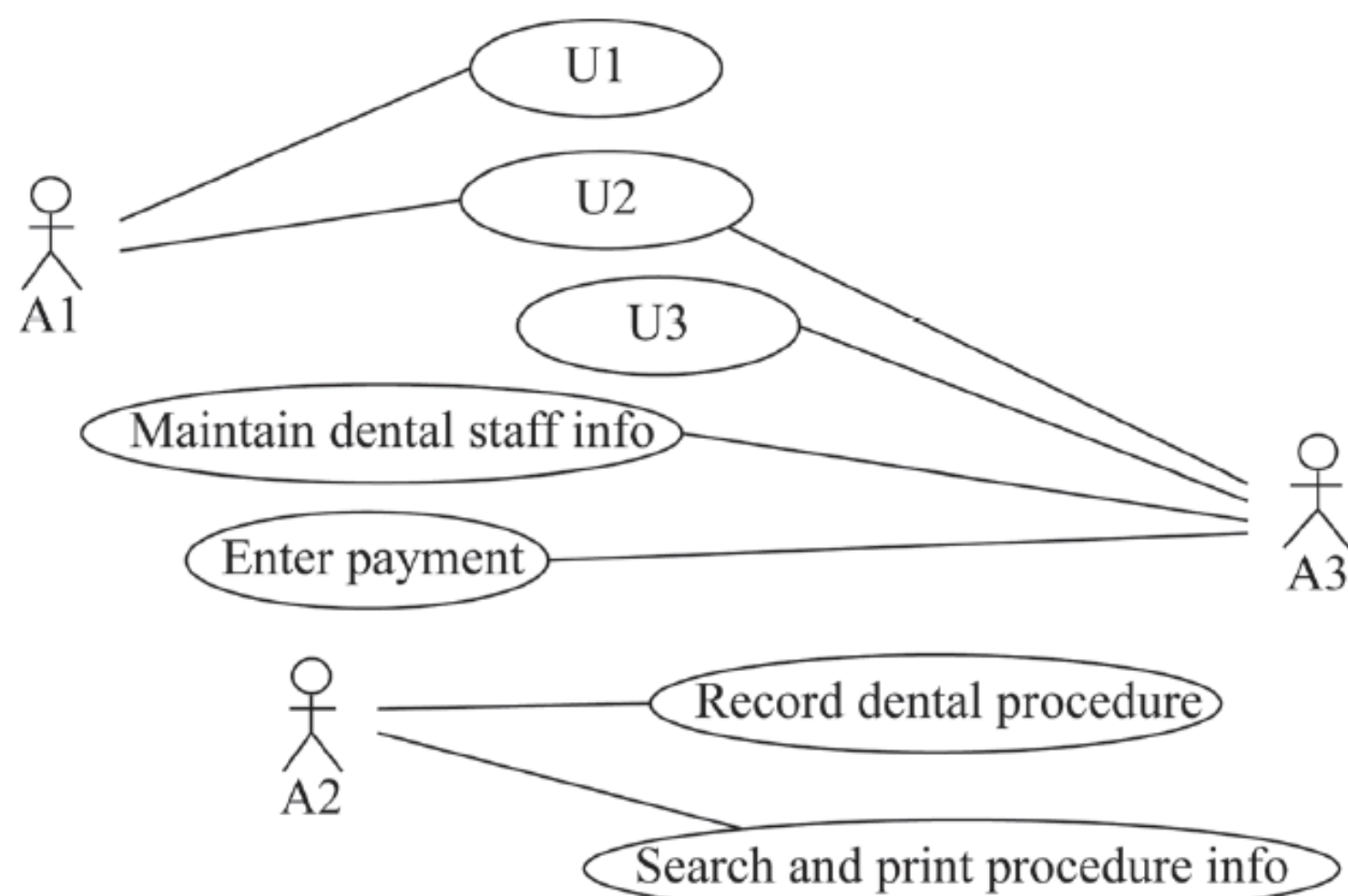


图 3-1 用例图

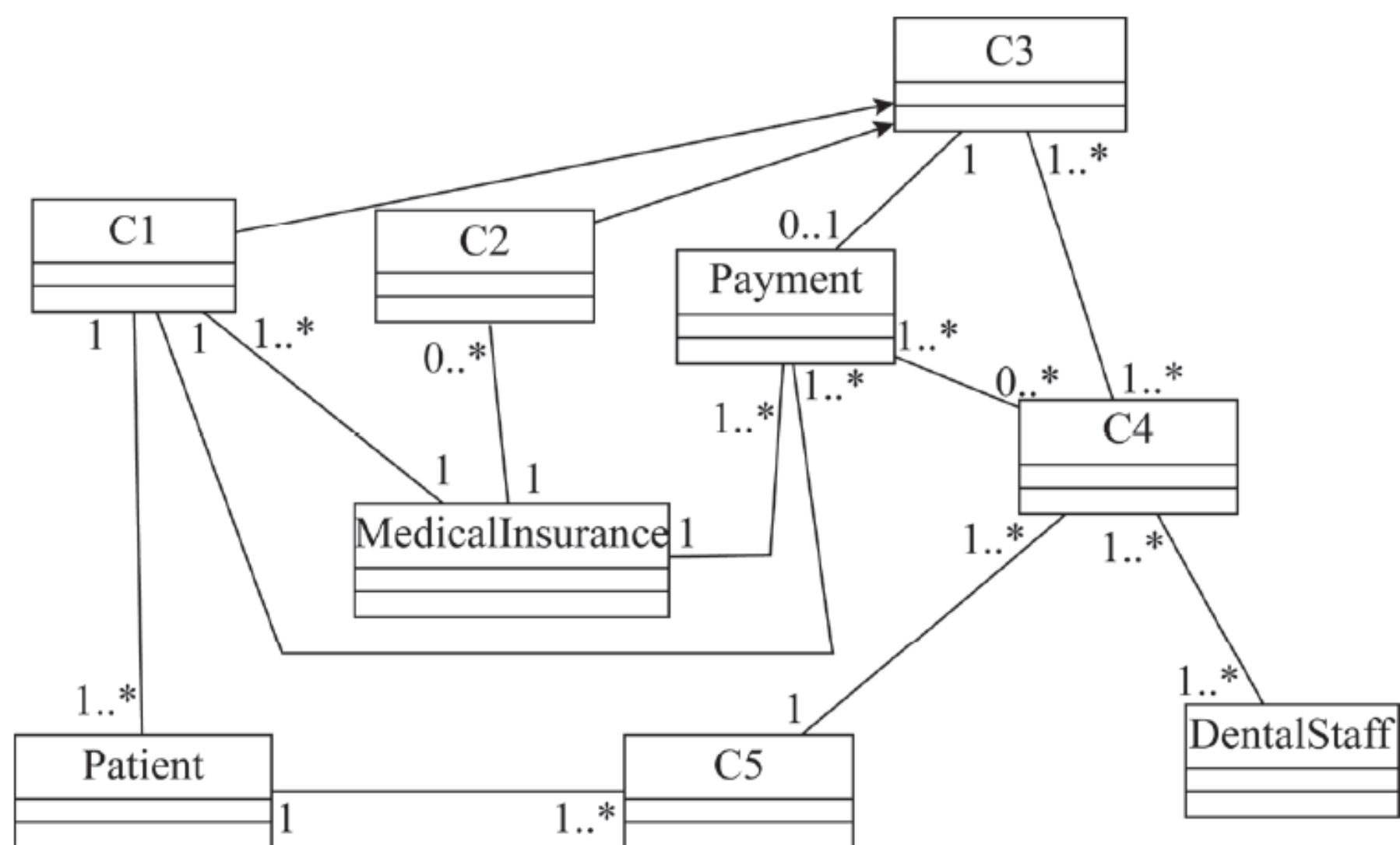


图 3-2 初始类图

试题三分析

本题主要考查面向对象分析与设计的基本概念。在建模方面，本题仅涉及了 UML 的用例图和类图，根据需求说明将模型补充完整。题目较为简单，属于经典考题。

【问题 1】

本问题给出的应用场景是一个用于管理病人基本信息和就诊信息的牙科诊所信息系统。

本问题要求补充用例图。用例图展现了一组用例、参与者以及它们之间的关系。用例建模是按照业务事件、谁发起事件，以及系统如何响应事件建模系统功能的过程。

参与者表示需要与系统交互以交换信息的任何事物。参与者可以是一个用户，可以是外部系统的一个角色，也可以是一个人。从题目的说明中可以很容易地发现，该系统中有 3 类与系统交互的角色：医护人员、接待员和办公人员。根据说明所描述的每个角

色所参与的功能，可以判断出：A1 对应的参与者是接待人员、A2 对应的参与者是医护人员、A3 对应的参与者是办公人员。

用例是一组相关行为的自动的和手动的步骤序列，其目的是完成单个业务任务。下面需要确定与参与者 A1（接待人员）相关联的用例。根据说明可知，A1 参与或激发的用例包括：记录病人基本信息和记录就诊记录，所以用例 U1 和 U2 分别对应“记录病人基本信息”和“记录就诊记录”。U3 是与参与者 A3（办公人员）相关联的用例。根据说明可知，A3 参与或激发的用例包括：打印发票、更新支付状态和记录医护人员信息。显而易见，U3 应该对应着用例会“打印发票”。

【问题 2】

本问题要求将图 3-2 所示的类图补充完整。

首先观察该类图，发现在图中存在一个继承结构：C3 为基类、C1 和 C2 为派生类。而这 3 个类又分别与类 Payment、MedicalInsurance 相关联，这两个类实现的都是与支付相关的功能。从说明中关于支付相关功能的描述可知，“发票分为两种：给医保机构的发票（InsuranceInvoice）和给病人的发票（PatientInvoice）”，这是一种明显的“一般——特殊”关系，这种关系在面向对象中通常采用继承机制来描述。表达一般概念的实体作为基类，特殊概念的实体作为派生类。由此可以确定 C3 对应类 Invoice。C1 关联着类 Patient，所以 C1 对应类 PatientInvoice，C2 对应类 InsuranceInvoice。

C4、C5 及其相关联的类，对应的是该系统维护就诊信息和治疗信息的功能。由说明可知，病人可以多次就诊；病人的每次就诊，可能包含多项治疗。这个需求由类 Patient、C5 及 C4 构成的关联结构表达。由此推断出，C5 对应着类 OfficeVisit，C4 对应着类 Procedure。

【问题 3】

类的属性可以从说明的相关描述中寻找。本题中只需找出类的关键属性即可，实现阶段所需添加的属性不需要考虑。

C4（Procedure）的属性描述在说明的第 3 条，包括：治疗项目名称、治疗项目描述、治疗的牙齿和费用。

C5（OfficeVisit）的属性描述在说明的第 2 条，包括：就诊时间、就诊费用、支付代码、病人支付费用、医保支付费用。

Patient 的属性描述在说明的第 1 条，包括：病人姓名、身份证号、出生日期、性别、首次就诊时间、最后一次就诊时间。

DentalStaff 的属性描述在说明的第 5 条，包括：姓名、职位、身份证号、家庭住址、联系电话。

参考答案

【问题 1】

A1: Receptionist 或接待员

A2: DentalStaff 或医护人员

A3: OfficeStaff 或办公人员

U1: Record office visit info 或记录就诊信息

U2: Maintain patient info 或记录病人基本信息

U3: Print invoices 或打印发票

【问题 2】

C1: PatientInvoice

C2: InsuranceInvoice

C3: Invoice

C4: Procedure

C5: OfficeVisit

【问题 3】

C4 的属性: 治疗项目名称、治疗项目描述、治疗的牙齿、费用。

C5 的属性: 就诊时间、就诊费用、支付代码、病人支付费用、医保支付费用。

Patient 的属性: 病人姓名、身份证号、出生日期、性别、首次就诊时间、最后一次就诊时间。

DentalStaff 的属性: 姓名、职位、身份证号、家庭住址、联系电话。

试题四 (共 15 分)

阅读下列说明和 C 代码, 回答问题 1 至问题 3, 将解答写在答题纸的对应栏内。

【说明】

0-1 背包问题定义为: 给定 i 个物品的价值 $v[1 \cdots i]$ 、重量 $w[1 \cdots i]$ 和背包容量 T , 每个物品装到背包里或者不装到背包里。求最优的装包方案, 使得所得到的价值最大。

0-1 背包问题具有最优子结构性质。定义 $c[i][T]$ 为最优装包方案所获得的最大价值, 则可得到如下所示的递归式。

$$c[i][T] = \begin{cases} 0 & \text{若 } i = 0 \text{ 或 } T = 0 \\ c[i-1][T] & \text{若 } T < w[i] \\ \max(c[i-1][T - w[i]] + v[i], c[i-1][T]) & \text{若 } i > 0 \text{ 且 } T \geq w[i] \end{cases}$$

【C 代码】

下面是算法的 C 语言实现。

(1) 常量和变量说明

T: 背包容量

v[]: 价值数组

w[]: 重量数组

c[][]: $c[i][j]$ 表示前 i 个物品在背包容量为 j 的情况下最优装包方案所能获得的最大价值

(2) C 程序

```
#include<stdio.h>
#include<math.h>
#define N 6
#define maxT 1000
int c[N][maxT]={0};

intMemoized_Knapsack(int v[N], int w[N], intT){
    inti;
    int j;
    for(i = 0;i< N;i++){
        for(j = 0;j <= T;j++){
            c[i][j] = -1;
        }
    }
    returnCalculate_Max_Value(v,w,N-1, T);
}

intCalculate_Max_Value(int v[N], int w[N],inti,int j){
    int temp = 0;
    if(c[i][j] != -1){
        return (1);
    }
    if(i == 0 || j == 0){
        c[i][j] = 0;
    }else{
        c[i][j] = Calculate_Max_Value(v,w,i-1,j);
        if((2)){
            temp = (3);
            if(c[i][j] < temp){
                (4);
            }
        }
    }
    return c[i][j];
}
```

【问题 1】(8 分)

根据说明和 C 代码, 填充 C 代码中的空 (1) ~ (4)。

【问题 2】(4 分)

根据说明和 C 代码, 算法采用了 (5) 设计策略。在求解过程中, 采用了 (6) (自

从表中可知 $c[5][11]=40$ 。

参考答案

【问题 1】

- (1) $c[i][j]$
- (2) $w[i] \leq j$
- (3) $\text{Calculate_Max_Value}(v, w, i-1, j-w[i]) + v[i]$
- (4) $c[i][j] = \text{temp}$

【问题 2】

- (5) 动态规划
- (6) 自顶向下

【问题 3】

- (7) 40

注意：从试题五和试题六中，任选一道题解答。

试题五（共 15 分）

阅读下列说明和 C++ 代码，将应填入 (n) 处的字句写在答题纸的对应栏内。

【说明】

某文件管理系统中定义了类 OfficeDoc 和 DocExplorer。当类 OfficeDoc 发生变化时，类 DocExplorer 的所有对象都要更新其自身的状态。现采用观察者（Observer）设计模式来实现该需求，所设计的类图如图 5-1 所示。

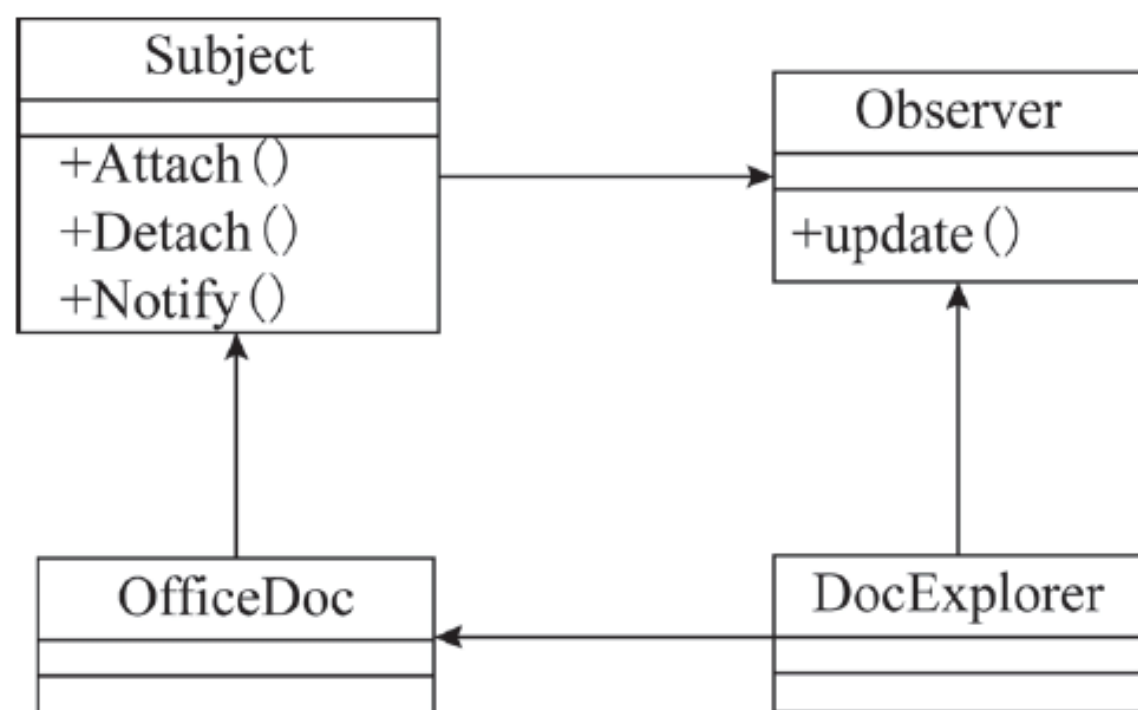


图 5-1 类图

【C++ 代码】

```
#include<iostream>
#include<vector>
#include<string>
using namespace std;

class Observer{
```



```
public:
    (1);
};

class Subject{
protected:
    vector<(2)>myObs;
public:
    virtual void Attach(Observer *obs) {myObs.push_back(obs);}
    virtual void Detach(Observer *obs) {
        for (vector<Observer*>::iterator iter = myObs.begin(); iter !=
            myObs.end(); iter++){
            if(*iter == obs){myObs.erase(iter); return;}
        }
    }
    virtual void Notify(){
        for (vector<Observer*>::iterator iter=myObs.begin(); iter!=myObs.end();
            iter++) {
            (3);
        }
    }
    virtual int getStatus() = 0;
    virtual void setStatus(int status) = 0;
};

class OfficeDoc : public Subject{
private:
    string mySubjectName;
    int m_status;
public:
    OfficeDoc(string name): mySubjectName(name), m_status(0){}
    void setStatus(int status){m_status = status;}
    int getStatus(){return m_status;}
};

class DocExplorer : public Observer{
private:
    string myObsName;
public:
    DocExplorer(string name, (4) sub): myObsName(name) {sub->(5);}
    void update(){cout<< "update observer:" <<myObsName<<endl;}
};
```



```

int main() {
    Subject *subjectA = new OfficeDoc("subject A");
    Observer *observerA = new DocExplorer("observerA", subjectA);
    subjectA->setStatus(1); subjectA->Notify();
    return 0;
}

```

试题五分析

本题考查设计模式中的观察者（Observer）模式的基本概念和应用。

观察者模式的意图是，定义对象间的一种一对多的依赖关系，当一个对象的状态发生改变时，所有依赖于它的对象都得到通知并被自动更新。观察者模式的结构图如图 5-2 所示。

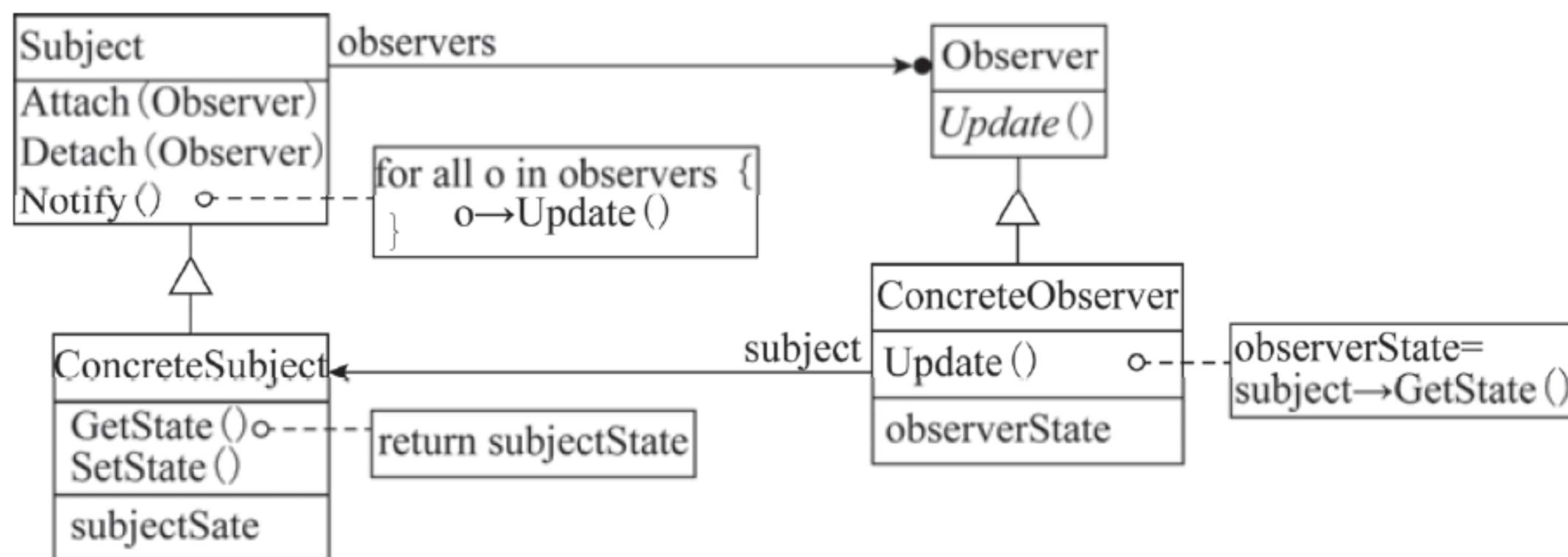


图 5-2 观察者模式结构图

其中：

- **Subject**（目标）知道它的观察者，可以有任意多个观察者观察同一个目标；提供注册和删除观察者对象的接口。
- **Observer**（观察者）为那些在目标发生改变时需获得通知的对象定义一个更新接口。
- **ConcreteSubject**（具体目标）将有关状态存入各 **ConcreteObserver** 对象；当它的状态发生改变时，向它的各个观察者发出通知。
- **ConcreteObserver**（具体观察者）维护一个指向 **ConcreteSubject** 对象的引用；存储有关状态，这些状态应与目标的状态保持一致；实现 **Observer** 的更新接口，以使自身状态与目标的状态保持一致。

观察者模式适用于：

- 当一个抽象模型有两个方面，其中一个方面依赖于另一个方面，将这两者封装在独立的对象中以使它们可以各自独立地改变和复用。
- 当对一个对象的改变需要同时改变其他对象，而不知道具体有多少对象有待改变时。

- 当一个对象必须通知其他对象，而它又不能假定其他对象是谁，即不希望这些对象是紧耦合的。

本题中的类 Subject 对应于图 5-2 中的 Subject，类 OfficeDoc 对应 ConcreteSubject；类 Observer 对应图 5-2 中的 Observer，类 DocExplore 对应 ConcreteObserver。DocExplore 维护一个指向 OfficeDoc 对象的引用，当 OfficeDoc 的状态发生改变时，向 DocExplore 发出通知。

第（1）空需要填写 Observer 中的核心方法，即在目标发生改变时通知观察者的更新接口。这个方法的原型可以在 Observer 的子类 DocExplore 中找到：`void update()`。update 方法需要在子类中进行重置，这里采用了 C++ 中的动态多态机制——纯虚拟函数。所以第（1）空应填入 `virtual void update() = 0`。

一个 Subject 可以有多个观察者，在 Subject 中需要提供增加和删除观察者的接口，即类中的 Attach、Detach 方法。这两个方法的主要操作对象就是类中的属性 myObs。根据程序上下文推断，myObs 表示的应该是观察者的集合，所以第（2）空应填入 `Observer*`。

第（3）空出现在 Subject 的方法 Notify 中，这个方法的功能是当目标发生变化时，通知所有与该目标关联的观察者，即调用每个观察者定义的 update 方法，所以第（3）空应填入 `(*iter) -> update()`。

DocExplore 是一个具体的观察者，它需要维护一个指向目标的对象，在这里实际上就是指向 OfficeDoc 的对象。观察者与目标的关联关系是通过 DocExplore 的构造函数实现的。在面向对象的继承机制中，通常倾向于用基类指针代替派生类指针，因此第（4）空应填入 `Subject*`。

观察者与目标的关联关系的建立需要调用 Subject 中的方法 Attach，因此第（5）空应填入 `Attach(this)`。

参考答案

- （1）`virtual void update() = 0`
- （2）`Observer *`
- （3）`(*iter)->update()`
- （4）`Subject*`
- （5）`Attach(this)`

试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入 （n） 处的字句写在答题纸的对应栏内。

【说明】

某文件管理系统中定义了类 OfficeDoc 和 DocExplorer。当类 OfficeDoc 发生变化时，类 DocExplorer 的所有对象都要更新其自身的状态。现采用观察者（Observer）设计模式来实现该需求，所设计的类图如图 6-1 所示。

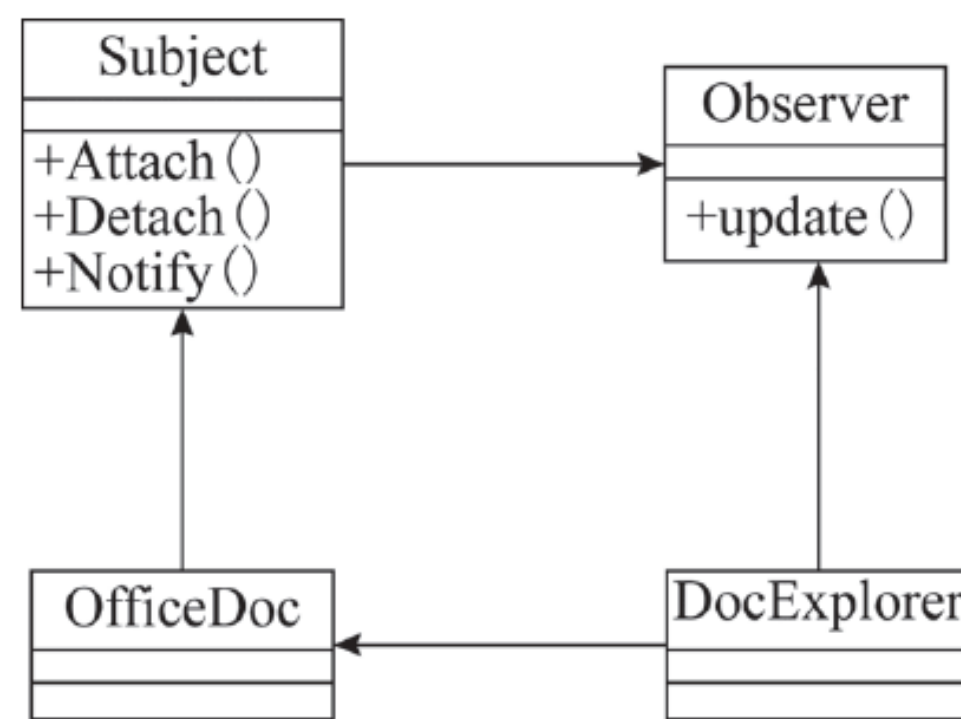


图 6-1 类图

【Java 代码】

```

import java.util.*;

interface Observer{
    public (1);
};

interface Subject{
    public void Attach(Observer obs);
    public void Detach(Observer obs);
    public void Notify();
    public void setStatus(int status);
    public int getStatus();
};

class OfficeDoc implements Subject{
    private List<(2)>myObs;
    private String mySubjectName;
    private int m_status;
    public OfficeDoc(String name){
        mySubjectName = name;
        this.myObs = new ArrayList<Observer>();
        m_status = 0;
    }
    public void Attach(Observer obs){ this.myObs.add(obs); }
    public void Detach(Observer obs){ this.myObs.remove(obs); }
    public void Notify(){
        for(Observer obs: this.myObs){ (3); }
    }
    public void setStatus(int status){
        m_status = status;
    }
}

```



```

        System.out.println("SetStatus subject[" + mySubjectName + "]status:"
            + status);
    }
    public int getStatus() {return m_status;}
};

class DocExplorer implements Observer{
    private String myObsName;
    public DocExplorer(String name, (4) sub){
        myObsName = name;
        sub.(5);
    }
    public void update() {
        System.out.println("update observer[" + myObsName + "]");
    }
};

class ObserverTest{
    public static void main(String[] args){
        Subject subjectA = new OfficeDoc("subject A");
        Observer observerA = new DocExplorer("observer A", subjectA);
        subjectA.setStatus(1);
        subjectA.Notify();
    }
}

```

试题六分析

本题考查设计模式中的观察者（Observer）模式的基本概念和应用。

观察者模式的意图是，定义对象间的一种一对多的依赖关系，当一个对象的状态发生改变时，所有依赖于它的对象都得到通知并被自动更新。观察者模式的结构图如图 6-2 所示。

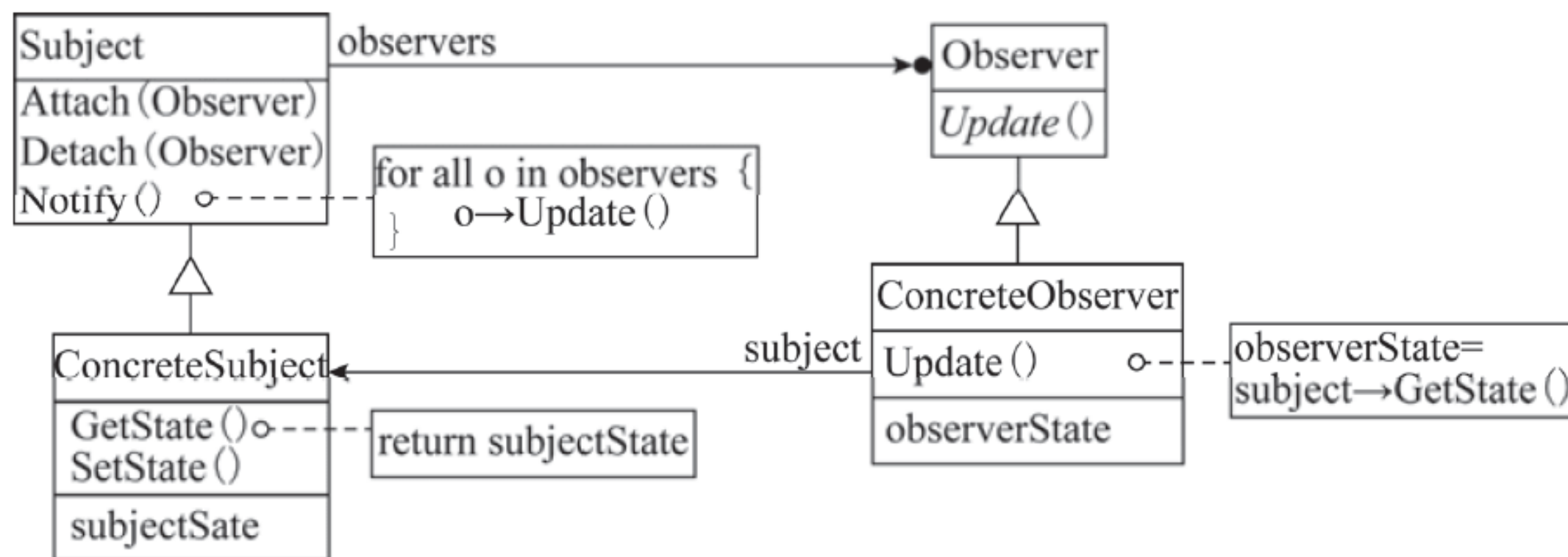


图 6-2 观察者模式结构图

其中:

- **Subject** (目标) 知道它的观察者, 可以有任意多个观察者观察同一个目标; 提供注册和删除观察者对象的接口。
- **Observer** (观察者) 为那些在目标发生改变时需获得通知的对象定义一个更新接口。
- **ConcreteSubject** (具体目标) 将有关状态存入各 **ConcreteObserver** 对象; 当它的状态发生改变时, 向它的各个观察者发出通知。
- **ConcreteObserver** (具体观察者) 维护一个指向 **ConcreteSubject** 对象的引用; 存储有关状态, 这些状态应与目标的状态保持一致; 实现 **Observer** 的更新接口, 以使自身状态与目标的状态保持一致。

观察者模式适用于:

- 当一个抽象模型有两个方面, 其中一个方面依赖于另一个方面, 将这两者封装在独立的对象中以使它们可以各自独立地改变和复用。
- 当对一个对象的改变需要同时改变其他对象, 而不知道具体有多少对象有待改变时。
- 当一个对象必须通知其他对象, 而它又不能假定其他对象是谁, 即不希望这些对象是紧耦合的。

本题中的类 **Subject** 对应于图 6-2 中的 **Subject**, 类 **OfficeDoc** 对应 **ConcreteSubject**; 类 **Observer** 对应图 6-2 中的 **Observer**, 类 **DocExplore** 对应 **ConcreteObserver**。**DocExplore** 维护一个指向 **OfficeDoc** 对象的引用, 当 **OfficeDoc** 的状态发生改变时, 向 **DocExplore** 发出通知。

第 (1) 空需要填写 **Observer** 中的核心方法, 即在目标发生改变时通知观察者的更新接口。这个方法的原型可以在 **Observer** 的子类 **DocExplore** 中找到: `void update()`。这里 **Observer** 被定义为了接口, `update` 方法需要在其子类中进行实现, 所以第 (1) 空应填入 `void update()`。

一个 **Subject** 可以有多个观察者, 在 **Subject** 中需要提供增加和删除观察者的接口, 即类中的 `Attach`、`Detach` 方法。这两个方法的主要操作对象就是类中的属性 `myObs`。根据程序上下文推断, `myObs` 表示的应该是观察者的集合, 所以第 (2) 空应填入 **Observer**。

第 (3) 空出现在 **Subject** 的方法 `Notify` 中, 这个方法的功能是当目标发生变化时, 通知所有与该目标关联的观察者, 即调用每个观察者定义的 `update` 方法, 所以第 (3) 空应填入 `obs.update()`。

DocExplore 是一个具体的观察者, 它需要维护一个指向目标的对象, 在这里实际上就是指向 **OfficeDoc** 的对象。观察者与目标的关联关系是通过 **DocExplore** 的构造函数实现的。因此第 (4) 空应填入 **Subject**。

观察者与目标的关联关系的建立需要调用 **Subject** 中的方法 **Attach**，因此第（5）空应填入 **Attach(this)**。

参考答案

- （1）void update()
- （2）Observer
- （3）obs.update()
- （4）Subject
- （5）Attach(this)